Lista 1. Konfiguracja projektu

October 13, 2026

Przykładowe rozwiazanie listy
https://github.com/mateuszpolec/uwr-pgier/tree/main/2026/l1.

1. **Struktura** W folderze projektu stwórz konfiguracje folderów, umożliwiajaca przechowywanie kodu gry, silnika, testów i bibliotek osobno. Rekomendowałbym podzielenie folderu z bibliotekami na trzy podfoldery – jeden dla dynamicznych bibliotek, drugi dla plików nagłówkowych i trzeci dla bibliotek statycznych.

2. **System budowania** Zależnie od wybranego systemu budowania stwórz trzy osobne "targety":

   (a) Biblioteke statyczna dla silnika.

   (b) Executable (aka. aplikacje konsolowa) dla gry, linkujaca biblioteke silnika.

   (c) Executable (aka. aplikacje konsolowa) dla testów, linkujaca biblioteke silnika.

3. **Pomocnicze skrypty** Na tym etapie stwórz dwa pomocnicze skrypty (lub trzy, jeżeli wybrałeś system budowania inny, niż premake5):

   (a) Generate - Tworzacy plik solucji/Tworzacy pliki make.

   (b) Clean - Czyszczacy repozytorium z metadanych i plików powiazanych z procesem kompilacji.

   (c) Compile* (tylko dla ludzi wybierajacych CMake) - Kompilujacy projekt.

4. Pobierz biblioteke SFML i pomyślnie zlinkuj do projektu silnika. https://www.sfml-dev.org/download/sfml/3.1.0/. Do sprawdzenia, czy wszystko działa, skorzystaj z gotowego przykładu dostarczonego przez SFML https://www.sfml-dev.org/documentation/3.1.0/, oczyść kod z zależności zasobów, których fizycznie nie ma. Efekt końcowy: Wyświetlone okno aplikacji graficznej.

5. Pobierz biblioteke nlohmann/json (https://github.com/nlohmann/json) i zaimportuj do projektu – jest to biblioteka nagłówkowa, w zwiazku z tym spodziewaj sie jedynie importu pliku `.h`.

6. Pobierz biblioteke spdlog (https://github.com/gabime/spdlog) i zaimportuj do projektu – jest to biblioteka nagłówkowa, w zwiazku z tym spodziewaj sie jedynie importu plików `.h`.

7. Pobierz biblioteke catch2 (https://github.com/catchorg/catch2) i zaimportuj do projektu. Biblioteka ta składa sie z jednego pliku nagłówkowego i jednego pliku źródłowego. Plik źródłowy powinien być umieszczony w wydzielonym folderze testów i dzielić przestrzeń folderu z plikiem **main.cpp**. Plik nagłówkowy umieść w folderze wydzielonym dla bibliotek (preferencyjnie podfolderze nagłówków).

8. Pobierz i skonfiguruj biblioteke imgui. W tym celu:

   (a) Pobierz biblioteke z głównego repozytorium https://github.com/ocornut/imgui.

   (b) Pobierz biblioteke wspierajaca rendering imgui w SFML: https://github.com/SFML/imgui-sfml.

   (c) Pobierz GLFW: https://www.glfw.org/download.

   (d) Biblioteke dynamiczna GLFW umieść w przeznaczonym na to podfolderze w folderze bibliotek.

   (e) Pliki nagłówkowe GLFW umieść w przeznaczonym na to podfolderze w folderze bibliotek.

   (f) Pliki nagłówkowe imgui umieść w przeznaczonym na to podfolderze w folderze bibliotek. Potrzebujesz jedynie:

      i. **backends/imgui_impl_glfw.h**
      ii. **misc/cpp/imgui_stdlib.h**
      iii. **misc/single_file/imgui_single_file.h**
      iv. I pozostałych plików powiazanych z imgui: **imconfig.h**, **imgui.h**, **imgui_internal.h**, **imstb_rectpack.h**, **imstb_textedit.h**, **imstb_textedit.h**.

   Cała reszta na tym etapie jest niepotrzebna i możesz ja pominać/usunać.

   (g) Pliki źródłowe imgui umieść w folderze przeznaczonym do kodu silnika (preferencyjnie utwórz osobny podfolder **imgui**). Potrzebujesz:

      i. **backends/imgui_impl_glfw.cpp**
      ii. **cpp/imgui_stdlib.cpp**
      iii. I pozostałych zależności imgui: **imgui.cpp**, **imgui_demo.cpp**, **imgui_draw.cpp**, **imgui_tables.cpp**, **imgui_widgets.cpp**.

   (h) Na tym etapie możliwe, że wymagane bedzie również zaktualizowanie ścieżek **include "imgui.h"**. Obserwuj błedy kompilacji i reaguj na bieżaco.

(i) Zrób dokładnie to samo dla biblioteki **Imgui-SFML**, proponuje umieścić folder z nagłówkami wspólnie z nagłówkami **imgui** i analogicznie zrobić z plikami źródłowymi.

(j) Możliwe, że w bibliotece **Imgui-SFML** pojawia sie błedy. W moim przypadku był to bład z assercja:

i. Jeden powiazany z assercja: `assert(io.Fonts->TexID != (ImTextureID) nullptr);`, która po prostu zakomentowałem.

9. Stwórz klase **CGame** w projekcie silnika, przenieś do tej klasy najważniejsze funkcjonalności:

(a) Tworzenie okna SFML

(b) Inicjalizacje ImGui.

(c) Metode uruchamiajaca nieskończona petle gry.

(d) Metode **Frame** lub **Update** lub **Tick**, a w niej obliczanie czasu trwania ostatniej klatki i procesowanie wydarzeń (eventów) z SFML.

(e) Metode przetwarzajaca eventy przychodzace z SFML.

(f) Metode **Render** lub **Draw** renderujaca dane do okna.

10. Include'uj klase **CGame** w projekcie gry, stwórz jej instancje i zawołaj metode odpowiedzialna za nieskończona petle gry.